

Programmierentwurf 1

Vorwort

Mit dem Nucleo-Board NUG474RE und dem EduShield sind eine Programmieraufgabe umzusetzen.

Die LED Reihe auf dem EduShield soll sequentiell aufleuchten. Mit einem Linear-Potentiometer sollen Werte von 09 bis 90 auf den 7 Segment-Anzeigen dargestellt werden. Nachfolgend sind die Anforderungen in Bezug auf Hardware, Software, Qualität und Lieferumfang aufgelistet. Sie dienen dazu einen definierten Programmentwurf abzugeben.

Hardware-Anforderung

HW1: Als Target Prozessor ist ein STM32G474RE auf einem Nucleo Board zu verwenden.

HW2: Als Ein- und Ausgabe Modul ist ein EduShield zu verwenden.

HW3: Die Binärdatei des Programmentwurfs ist ablauffähig auf dem Nucleo Board in Verbindung mit dem EduShield zu erstellen.

HW4: Es sind keine weiteren elektronischen Schaltungen für den Programmentwurf zulässig.

Software-Anforderung

SW1: Die Ausgangsbasis für den Programmentwurf ist das Programmieraufgabe_Basis.zip Projekt.

SW2: Der wiederkehrende Teil des Programmentwurfs ist im Kontext der Funktion RunState_Cyclic_Running in run_state.c aufzurufen. Die Initialisierung der Programmmodule in RunState_Init() durchzuführen und die Initialisierung der Peripherie in der main() vor Aufruf der MainState_Init().

SW3: Alle nachfolgenden Funktionen sind so auszuführen, dass sie zu jedem Zeitpunkt unabhängig voneinander ausgeführt werden können.

LEDs auf dem EduShield

SW4: Für die Dauer des Programmablaufs leuchten die LEDs D4 bis D0 auf dem EduShield in sequentieller Reihenfolge [D4, D3, D2, D1, D0] für jeweils 1 Sekunde pro LED auf.

SW5: Nachdem die LED D0 erloschen ist, leuchtet die LED D4 auf. Das Umschalten zwischen den LEDs muss in weniger als der Zykluszeit von 1 ms durchgeführt werden.

SW6: Unmittelbar bei Betätigung des blauen User Buttons B1 auf dem Nucleo Board soll die Leuchtsequenz aus Anforderung <SW4> starten. Die Leuchtsequenz soll bei LED D2 beginnen. Initial beim Hochfahren soll die Leuchtsequenz aus sein. Wenn die Leuchtsequenz bereits läuft, soll diese durch den User Button B1 nicht beeinflusst werden.

SW7: Bei Betätigung des Tasters SW1 auf dem Nucleo Board sollen alle LEDs ausgehen. Die Leuchtsequenz endet damit umgehend.

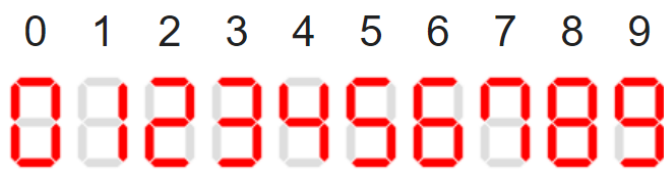
7-Segmentanzeige

SW10: Für die Konvertierung von Spannungsmessungen in diesem Kapitel sollen geeignete Maßnahmen in Hardware getroffen werden, um den vollen Konvertierungsbereich von Masse bis VREF+ auszuschöpfen

SW11: Der variable Spannungswert des linken Linearpotentiometers R1 auf dem EduShield soll in eine Zahl mit einer Auflösung von 8-Bit konvertiert werden.

SW12: Die Zahl aus Anforderung <SW11> soll linear in den Ganzzahlen-Wertebereich von 0 bis 9 konvertiert werden. Die Randwerte 0 und 9 sollen durch jeweils genau 28 Rohwerte erreicht werden können.

SW13: Die Ganzzahlen aus Anforderung <SW12> sollen folgendermaßen auf die linke 7-Segmentanzeige S1 dargestellt werden:



SW14: Auf der rechten 7-Segmentanzeige S2 sollen Ganzzahlen im Allgemeinen ebenso wie in dem Bild oben dargestellt werden. Die Summe der dargestellten Werte auf S1 und S2 soll stets 9 ergeben.

<INFO1>: Aufleuchtende Segmente werden nachfolgend als rote Segmente bezeichnet, die anderen als graue Segmente.

SW15: Alle roten Segmente der zwei 7-Segmentanzeigen sollen in der gleichen Helligkeit aufleuchten. Die roten Segmente dürfen nicht flackern, außer während des Übergangs zwischen zwei Ganzzahlen <SW12>.

SW16: Der variable Spannungswert des rechten Linearpotentiometers R2 auf dem EduShield soll in eine Zahl mit einer Auflösung von 8-Bit konvertiert werden.

SW17: Der Rohwert aus <SW16> soll für das Dimmen der zwei 7-Segmentanzeigen verwendet werden. Die Funktion des Dimmens soll durch ein PWM Signal gewährleistet werden, welches direkt durch eine Timer-Peripherie generiert wird.

SW18: Die Umschaltzeitpunkte zwischen den zwei 7-Segmentanzeigen sollen stets mit dem Ende einer Pulsweitenperiode zusammenfallen.

SW19: Der Rohwert aus <SW16> soll quadratisch auf das PWM Signal abgebildet werden.

Qualitäts-Anforderung

Q1: Der Programmentwurf soll in der Sprache C (C99) abgefasst werden.

Q2: Der Quellcode soll für eine ARM GNU Toolchain ab Version 12.3.Rel1 kompilier- und linkfähig sein.

Q3: Die C- und H- Dateien im Projektordner /source sollen mit Astyle formatiert werden.

Q4: Der Programmteil aus dem Kapitel „LEDs auf dem Edushield“ soll eine öffentliche Initialisierungsfunktion (edushield_init) und eine öffentliche, zyklische Funktion (edushield_cyclic) beinhalten. Der Programmcode aus diesem Kapitel soll vollständig in der Datei edushield.c/.h implementiert sein.

Q5: Der Programmteil aus dem Kapitel „7-Segmentanzeige soll eine öffentliche Initialisierungsfunktion (segment_init) und eine öffentliche, zyklische Funktion (segment_cyclic) beinhalten. Der Programmcode aus diesem Kapitel soll vollständig in der Datei segment.c/.h implementiert sein.

Q6: Die C-Dateien sollen dem Symbol C_SOURCES in der FileList.mak hinzugefügt werden.

Hinweis: C_SOURCE ist eine Liste, wobei jede Zeile zwischen zwei Pfaden mit einem Backslash \ beendet wird.

Q7: Die eigenen Algorithmen sollen in Englischer Sprache nachvollziehbar dokumentiert werden.

Q8: Doxygen gemäß Ausgangsbasis Anforderung <SW1> darf keine Warnungen erzeugen.

Q9: In Software darf nicht mehr umgesetzt sein als durch die Anforderungen des Programmentwurfs gefordert ist.

Q10: Das Programm soll ablauffähig auf dem Nucleo Board im Auslieferungszustand sein.

Q11: Der Programmentwurf ist eigenständig zu konzeptionieren, zu programmieren, zu bauen und gemäß Lieferanforderungen gebündelt abzugeben. Insbesondere ist die Verwendung von generativer künstlicher Intelligenz (z.B. ChatGPT) nicht zulässig. Darüber hinaus darf Quelltext nicht zwischen Teilnehmern der Prüfung geteilt werden. Davon abweichend dürfen alle Aufgaben im Kontext der Vorlesung benutzt werden.

Q12: Peripherie soll mit CubeMX konfiguriert werden.

Q13: Der generierte Code aus CubeMX soll zur Konfiguration der Peripherie in der main()-Funktion vor der MainState_Init() aufgerufen werden.

Lieferumfang und Abgabetermin

L1: Die Abgabe des Programmentwurfs soll in Form eines ZIP-Archivs erfolgen.

L2: Das Archiv aus L1 soll den Quellcode und die Projektverwaltung enthalten.

L3: Das Archiv aus L1 soll zusätzlich die Ausgabe des Build-Prozesses enthalten.

L4: Der Programmentwurf soll spätestens zum Datum der Abgabe vollständig abgegeben werden. Temporäre Abgaben zählen als Abgaben sofern keine finale Abgabe erfolgt ist. Die Möglichkeit einer Nachabgabe sofern final abgegeben wurde, liegt im Ermessen des Dozenten.

L5: Die Dateien aus L3 sollen funktional reproduzierbar sein durch den Inhalt aus L2.